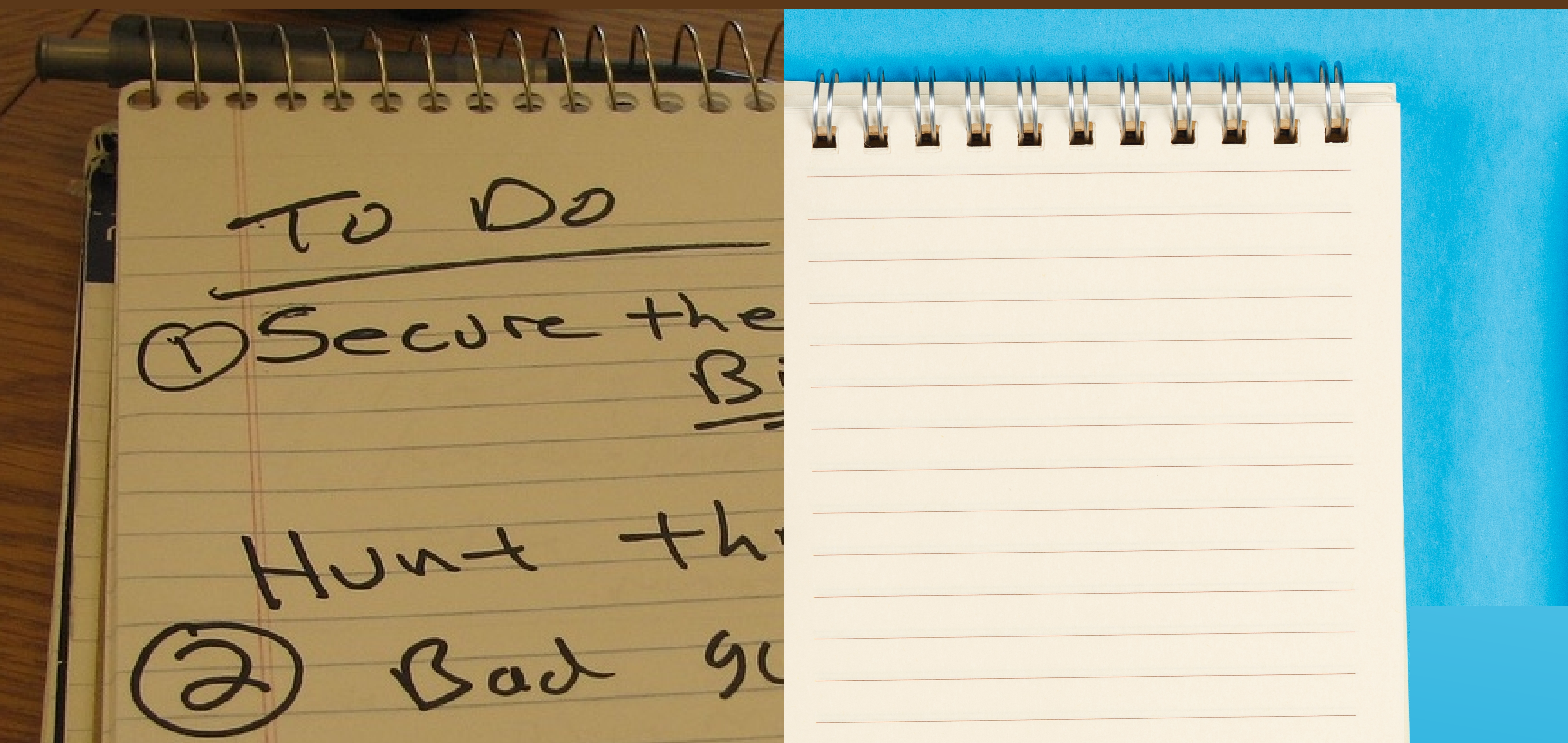


ToDo & Co

AUTHENTIFICATION DE L'APPLICATION

Documentation technique



2026



ToDo & Co

INTRODUCTION ET ARCHITECTURE GÉNÉRALE

Sommaire

 Introduction

	Introduction	15
	Parcours	16

 Vue d'ensemble

 Parcours

	Page 1	17
	Page 2	18

 Fichiers concernés

	Fichiers	19
	Utilisation	20



ToDo & Co

LES UTILISATEURS ET LEUR STOCKAGE

Sommaire

Entité User

	Utilisation	21
	Parcours	22

Identifiant de l'utilisateur

	Utilisation	23
	Parcours	24

Stockage des utilisateurs

	Utilisation	
	Page 1	26
	Page 2	26
	Parcours	27



ToDo & Co

LES UTILISATEURS ET LEUR STOCKAGE

Sommaire

Association avec les tâches

	Utilisation	28
	Parcours	29

Utilisateur anonymous

	Utilisation	30
	Parcours	31



ToDo & Co

LE PROCESSUS D’AUTHENTIFICATION

Sommaire

Configuration de Symfony Security

	 Configuration	32
	 Rôle	33

Fournisseur d’utilisateurs

	 Configuration	34
	 Parcours	35

Page de connexion

Utilisation

	Page 1	36
	Page 2	37

Parcours

	Page 1	38
	Page 2	39



ToDo & Co

LE PROCESSUS D’AUTHENTIFICATION

Sommaire

Formulaire de connexion

Formulaire

Page 1	40
Page 2	41

Fonctionnement du formulaire

Parcours

Page 1	42
Page 2	43

Firewall

 Configuration	44
 Parcours	45



ToDo & Co

LE PROCESSUS D’AUTHENTIFICATION

Sommaire

Déconnexion

	 Déconnexion	46
	 Parcours	47



ToDo & Co

PROTECTION DES MOTS DE PASSE

Sommaire

Hashage des mots de passe

	 Protection	48
	 Utilisation	49
	 Parcours	50

Mot de passe dans l'entité User

	 Utilisation	51
	 Protection	52



ToDo & Co

RÔLES ET CONTRÔLE DES ACCÈS

Sommaire

Gestion des rôles

	Rôles	53
	Fonctionnement	54

Modification du rôle

	Rôles	55
	Validation	56

Contrôle des accès

	Configuration	57
	Parcours — Accès administrateur	58
	Parcours — Accès utilisateur	59



ToDo & Co

RÔLES ET CONTRÔLE DES ACCÈS

Sommaire

Accès à /login

	 Route publique	60
	 Parcours	61

Gestion des utilisateurs

	 Administrateur	62
	 Autorisation	63

Accès aux autres pages

	 Utilisateur	64
	 Accès	65



ToDo & Co

AUTORISATION DES ACTIONS SUR LES TÂCHES

Sommaire

Authentification vs autorisation

	Authentification	66
	Autorisation	67
	Parcours	68

TaskVoter

	Fonctionnement	69
	Décision	70



ToDo & Co

AUTORISATION DES ACTIONS SUR LES TÂCHES

Sommaire



Règle pour anonymous

	Utilisateur anonyme	71
	Règle spéciale	72



Règle pour anonymous

	Utilisateur anonyme	73
---	---------------------	----



ToDo & Co

FAIRE ÉVOLUER LA SÉCURITÉ DE L'APPLICATION

Sommaire

Modifier le comportement de connexion

	Fichiers  Évolution	74
---	--	----

Modifier les informations d'un utilisateur

	Fichiers  Évolution	75
---	--	----

Modifier les rôles disponibles

	Fichiers  Évolution	76
---	--	----

Modifier les autorisations sur les tâches

	Fichiers  Évolution	77
---	--	----

Modifier les accès aux routes

	Fichiers  Évolution	78
---	--	----



ToDo & Co

TESTS ET VÉRIFICATION DE LA SÉCURITÉ

Sommaire



Tests fonctionnels



Cas testés

Page 1

79

Page 2

80



Tests du TaskVoter



Cas testés

81



Autorisations

82



Exécution des tests



Commande

83



Résultat

84



ToDo & Co

RÉSUMÉ POUR UN NOUVEAU DÉVELOPPEUR

Sommaire

À retenir

1	 User.php	85
2	 security.yaml	86
3	 SecurityController.php	87
4	 login.html.twig	88
5	 UserPasswordHasherInterface	89
6	 access_control	90
7	 TaskVoter	91

Parcours global

 Général	92
 Règles de sécurité	93

Évolution de la sécurité

Page 1	94
Page 2	95
Page 3	96



ToDo & Co

INTRODUCTION ET ARCHITECTURE GÉNÉRALE

Introduction

Introduction

L’authentification permet à l’application d’identifier l’utilisateur qui souhaite accéder aux fonctionnalités de ToDo & Co.

Symfony Security prend en charge le processus d’authentification tandis que les règles d’autorisation déterminent ensuite les actions que l’utilisateur est autorisé à effectuer.



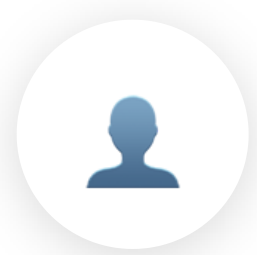
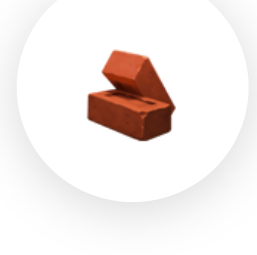
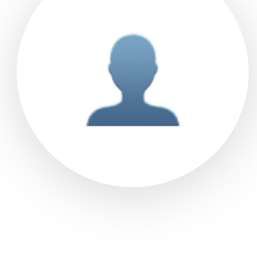
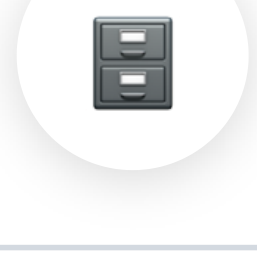
ToDo & Co

INTRODUCTION ET ARCHITECTURE GÉNÉRALE

 Introduction

-  Introduction
-  **Parcours**

 Parcours

Utilisateur		
	↓	
Formulaire de connexion		
	↓	
Symfony Security		
	↓	
Provider Doctrine		
	↓	
Entité User		
	↓	
Base de données		
	↓	
Utilisateur authentifié		



ToDo & Co

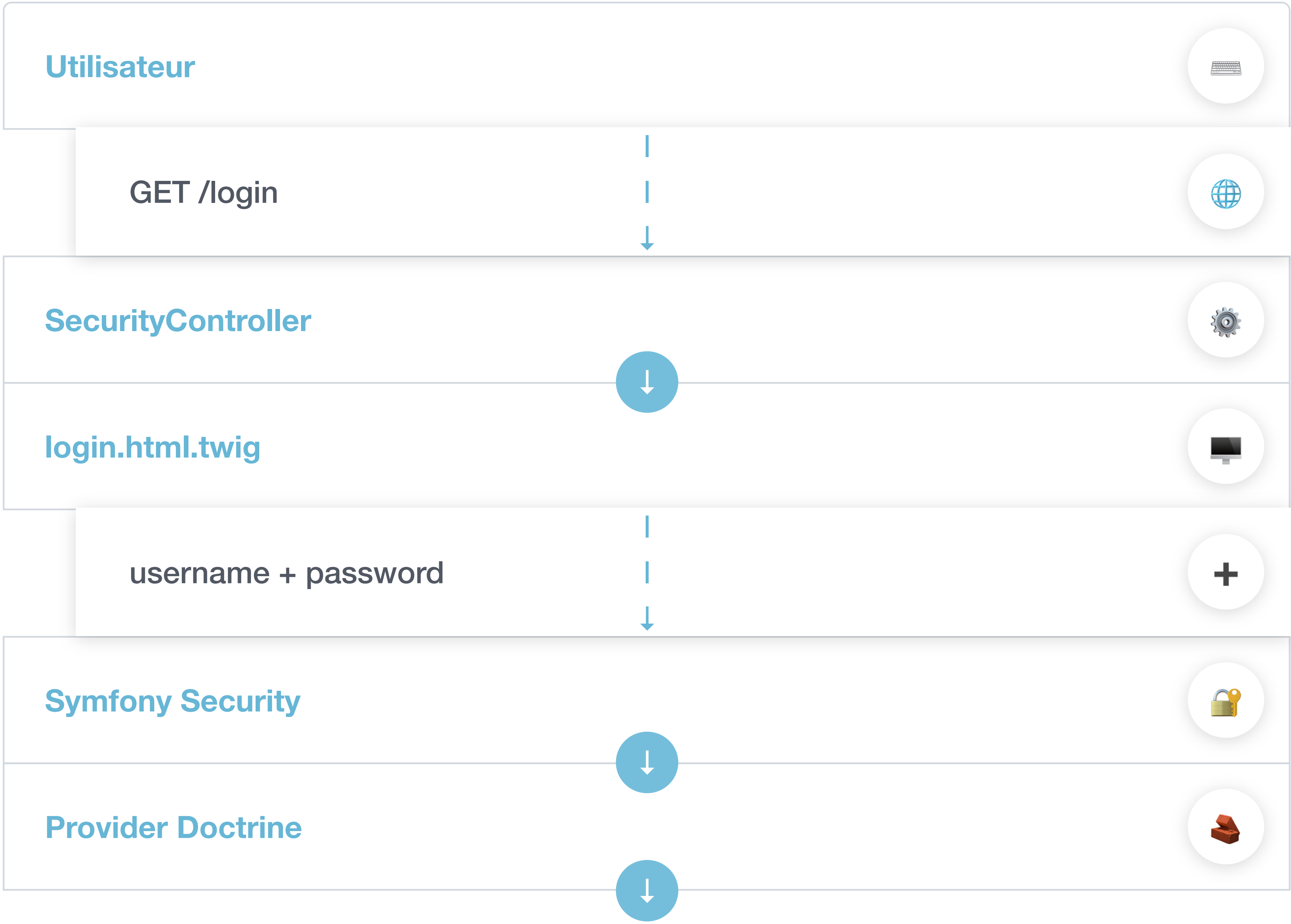
INTRODUCTION ET ARCHITECTURE GÉNÉRALE

 Vue d'ensemble

 Parcours

 Parcours

Le fonctionnement général de l'authentification est le suivant :





ToDo & Co

INTRODUCTION ET ARCHITECTURE GÉNÉRALE

 Vue d'ensemble

 Parcours

 Parcours

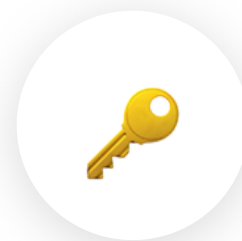
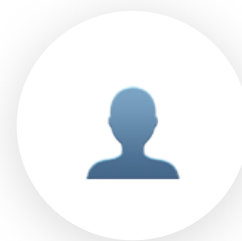
App\Entity\User

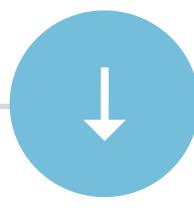
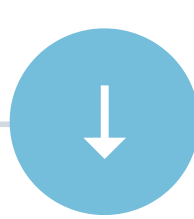
recherche par username

Base de données

Vérification du mot de passe

Utilisateur authentifié





Une fois authentifié, Symfony conserve l'état de l'utilisateur dans la session et utilise ses rôles pour déterminer les fonctionnalités auxquelles il peut accéder.

18



ToDo & Co

INTRODUCTION ET ARCHITECTURE GÉNÉRALE

 Fichiers concernés

-  Fichiers
-  Utilisation

 Fichiers

Les principaux fichiers intervenant dans l’authentification sont :

```
config/
├── packages/
│   └── security.yaml
src/
├── Controller/
│   └── SecurityController.php
├── Entity/
│   └── User.php
templates/
├── security/
│   └── login.html.twig
```



ToDo & Co

INTRODUCTION ET ARCHITECTURE GÉNÉRALE

 Fichiers concernés

-  Fichiers
-  Utilisation

 Utilisation

Chaque fichier possède un rôle différent.

Fichier	Rôle
security.yaml	Configuration de Symfony Security
User.php	Représentation d'un utilisateur et implémentation des interfaces Symfony
SecurityController.php	Affichage de la page de connexion
login.html.twig	Formulaire de connexion



ToDo & Co

LES UTILISATEURS ET LEUR STOCKAGE

Entité User

 Utilisation Parcours

Utilisation

L'entité utilisateur se trouve dans :

```
src/entity/user.php
```

Elle implémente deux interfaces Symfony :

```
userinterface  
passwordauthenticateduserinterface
```

La déclaration est :

```
class user implements userinterface, passwordauthenticateduserinterface
```

Ces interfaces permettent à l'entité d'être utilisée directement par le composant Security de Symfony.



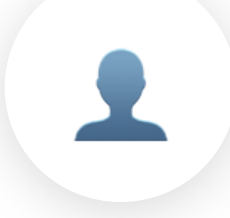
ToDo & Co

LES UTILISATEURS ET LEUR STOCKAGE

 Entité User

- Utilisation
-  Parcours

 Parcours

Username saisi		
	↓	
Provider Doctrine		
	↓	
App\Entity\User		
	↓	
getUserIdentifiant()		
	↓	
Username		
	↓	
Base de données		
	↓	
Utilisateur trouvé		



ToDo & Co

LES UTILISATEURS ET LEUR STOCKAGE

ID Identifiant de l'utilisateur

[Utilisation](#)[Parcours](#)

Utilisation

Symfony utilise la méthode :

```
getuseridentifiant()
```

pour obtenir l'identifiant de l'utilisateur.

Dans l'application :

```
public function getuseridentifiant(): string  
{  
    return (string) $this->username;  
}
```

L'identifiant retourné est donc le :

```
username
```

 **ToDo & Co** LES UTILISATEURS ET LEUR STOCKAGE

 Identifiant de l'utilisateur

- Utilisation
-  **Parcours**

 **Parcours**

Username		
getUserIdentifier()		
Identifiant Symfony		



ToDo & Co

LES UTILISATEURS ET LEUR STOCKAGE

 Stockage des utilisateurs

- Utilisation
- Parcours

 Utilisation

Les utilisateurs sont stockés dans la base de données via l'entité :

```
app\entity\user
```

La table correspondante est :

```
user
```

Elle est explicitement définie dans l'entité :

```
#[orm\table(name: 'user')]
```

Les principales informations stockées sont :

```
user
├── id
├── username
├── password
├── email
└── roles
```



ToDo & Co

LES UTILISATEURS ET LEUR STOCKAGE

 Stockage des utilisateurs

- Utilisation
- Parcours

 Utilisation

Les rôles sont stockés dans une colonne JSON :

```
#[orm\column(type: 'json')]  
private $roles = [];
```

Un utilisateur peut donc avoir une valeur telle que :

```
[<< ROLE_USER >>]
```

ou :

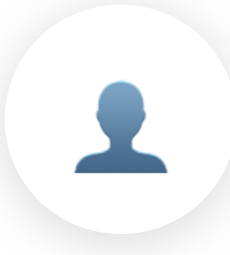
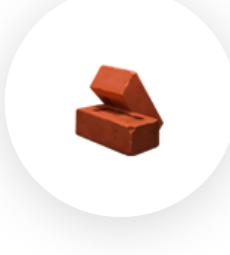
```
[<< ROLE_ADMIN >>]
```

 **ToDo & Co** LES UTILISATEURS ET LEUR STOCKAGE

 Stockage des utilisateurs

- Utilisation
-  **Parcours**

 **Parcours**

App\Entity\User	↓	
Entity Doctrine	↓	
Table user	↓	
Données utilisateur	↓	



ToDo & Co

LES UTILISATEURS ET LEUR STOCKAGE

Association avec les tâches

 Utilisation

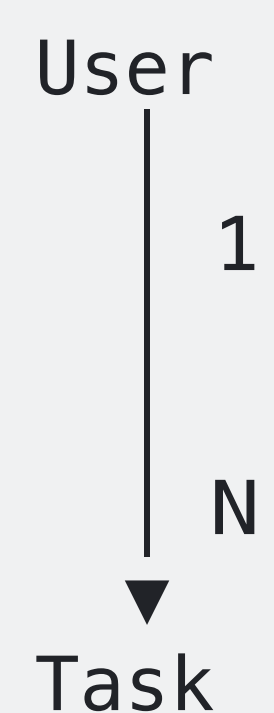
 Parcours

Utilisation

L'utilisateur possède également une relation avec les tâches :

```
#[orm\onetomany(mappedby: 'user', targetentity: task::class)]
private collection $tasks;
```

La relation est donc :



Lorsqu'une tâche est ajoutée à un utilisateur avec :

```
$user->addTask($task);
```

l'utilisateur est automatiquement défini comme propriétaire :

```
$task->setUser($this);
```

 **ToDo & Co** LES UTILISATEURS ET LEUR STOCKAGE

Association avec les tâches

- Utilisation
-  **Parcours**

Parcours

Le principe est donc :



Cela permet de maintenir la cohérence de l'association entre les deux entités.



ToDo & Co

LES UTILISATEURS ET LEUR STOCKAGE

 Utilisateur anonymous

- Utilisation
- Parcours

 Utilisation

Les anciennes tâches créées avant l’association obligatoire à un utilisateur sont rattachées à un utilisateur spécifique :

anonymous

Cet utilisateur possède un compte dédié dans la table user.



ToDo & Co

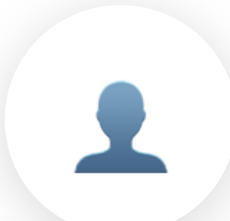
LES UTILISATEURS ET LEUR STOCKAGE

 Utilisateur anonymous

- Utilisation
-  **Parcours**

 Parcours

Son utilisation permet de conserver les anciennes tâches tout en maintenant désormais une association obligatoire :

Ancienne Task		
		
Migration		
		
anonymous		
		
Task.user		

Les utilisateurs standards ne peuvent pas supprimer ces tâches.

Les administrateurs peuvent les supprimer conformément aux règles définies par le TaskVoter.



ToDo & Co

LE PROCESSUS D’AUTHENTIFICATION

Configuration de Symfony Security

-  Configuration
-  Rôle

Configuration

La configuration se trouve dans :

```
config/packages/security.yaml
```



ToDo & Co

LE PROCESSUS D'AUTHENTIFICATION

 Configuration de Symfony Security

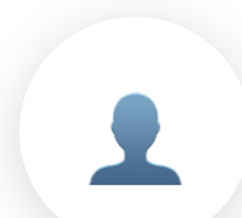
-  Configuration
-  **Rôle**

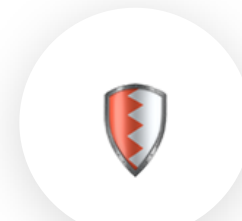
 Rôle

Elle définit notamment :

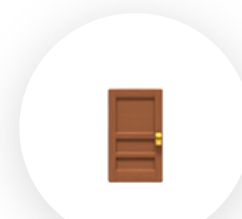
- le système de hashage des mots de passe

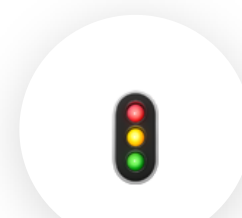

- le fournisseur d'utilisateurs


- le firewall


- le formulaire de connexion


- la déconnexion


- les règles d'accès aux différentes routes



La configuration de Security constitue donc le point central du système d'authentification.



ToDo & Co

LE PROCESSUS D'AUTHENTIFICATION

Fournisseur d'utilisateurs

 Configuration

 Parcours

Configuration

La configuration utilise le provider Doctrine :

```
providers:
  doctrine:
    entity:
      class: App\Entity\User
      property: username
```

Cela signifie que Symfony utilise l'entité :

```
App\Entity\User
```

pour retrouver les utilisateurs.

La recherche est effectuée à partir de la propriété :

```
username
```



ToDo & Co

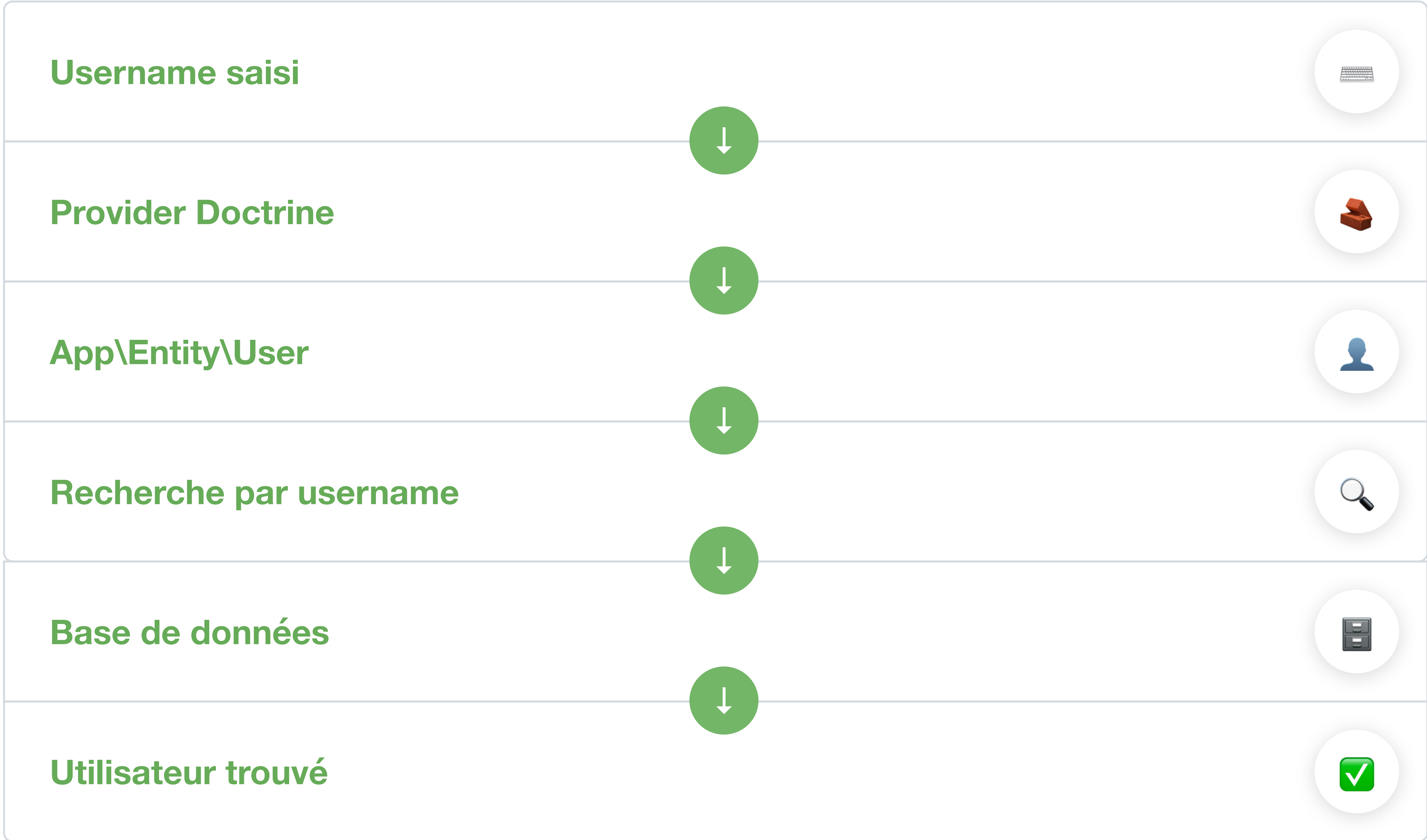
LE PROCESSUS D'AUTHENTIFICATION

 Fournisseur d'utilisateurs

- Configuration
-  Parcours

 Parcours

Le parcours est donc :



Le username est donc l'identifiant utilisé lors de la connexion.



ToDo & Co

LE PROCESSUS D'AUTHENTIFICATION

Page de connexion

[Utilisation](#)[Parcours](#)

Utilisation

La page de connexion est gérée par :

```
src/Controller/SecurityController.php
```

pour obtenir l'identifiant de l'utilisateur.

La route est :

```
#[Route('/login', name: 'login')]
```

La méthode login() récupère les informations nécessaires auprès de :

```
AuthenticationUtils
```

Elle transmet au template :

```
[  
    'last_username' => $authenticationUtils->getLastUsername(),  
    'error' => $authenticationUtils->getLastAuthenticationError(),  
]
```



ToDo & Co

LE PROCESSUS D’AUTHENTIFICATION

 Page de connexion

-  Utilisation
-  Parcours

 Utilisation

Cela permet notamment d’afficher :

le dernier nom d'utilisateur saisi



le message d'erreur lorsqu'une authentification échoue



Le contrôleur est donc principalement responsable de l’affichage de la page de connexion.



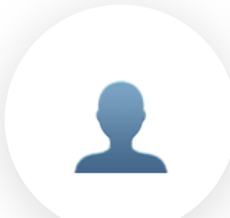
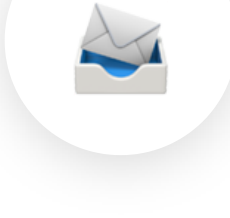
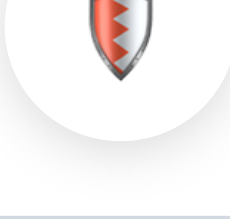
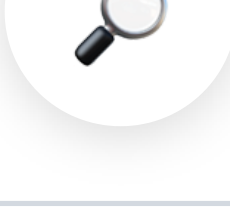
ToDo & Co

LE PROCESSUS D’AUTHENTIFICATION

 Page de connexion

- Utilisation
-  **Parcours**

 **Parcours**

Utilisateur		
	↓	
GET /login		
	↓	
SecurityController		
	↓	
login.html.twig		
	↓	
Username + password		
	↓	
POST /login		
	↓	
Symfony Security		
	↓	
Provider Doctrine		
	↓	



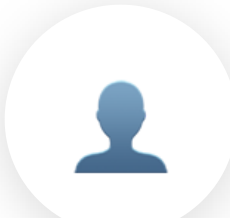
ToDo & Co

LE PROCESSUS D’AUTHENTIFICATION

 Page de connexion

- Utilisation
-  **Parcours**

 **Parcours**

User		
		
Vérification du mot de passe		
		
Authentification réussie		



ToDo & Co

LE PROCESSUS D’AUTHENTIFICATION

Formulaire de connexion

Formulaire

Formulaire

Le formulaire est défini dans :

```
templates/security/login.html.twig
```

pour obtenir l’identifiant de l’utilisateur.

Il utilise :

```
<form action=« {{ path('login') }} » method=« post »></form>
```

Les champs envoyés à Symfony sont :

```
_username  
_password
```

Le nom d’utilisateur est envoyé avec :

```
<input type=« text » id=« username » name=« _username » />
```



Formulaire de connexion

Formulaire

Formulaire

et le mot de passe avec :

```
<input type=« password » id=« password » name=« _password » />
```

Le formulaire contient également un token CSRF :

```
<input  
  type=« hidden »  
  name=« _csrf_token »  
  value=« {{ csrf_token('authenticate') }} »  
>
```

Ce token correspond à la configuration :

```
enable_csrf: true  
csrf_token_id: authenticate
```



ToDo & Co

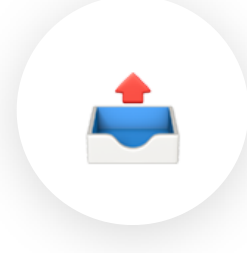
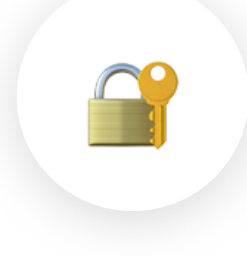
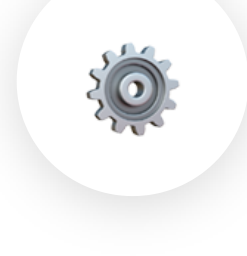
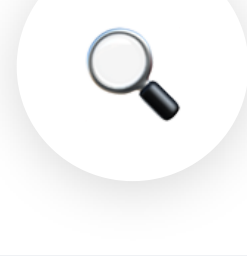
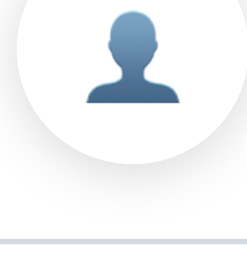
LE PROCESSUS D’AUTHENTIFICATION

 Fonctionnement du formulaire

 Parcours

 Parcours

Lorsqu’un utilisateur valide le formulaire :

Utilisateur		
	↓	
POST /login		
	↓	
Symfony Security		
	↓	
Provider Doctrine		
	↓	
Recherche du username		
	↓	
Utilisateur trouvé		
	↓	

🔄

Fonctionnement du formulaire

🗺️

Parcours

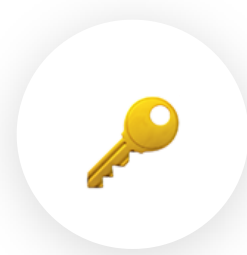
🗺️

Parcours

Vérification du mot de passe

↓

Authentification





Il est important de noter que SecurityController n’effectue pas lui-même la vérification du mot de passe.

Le contrôleur sert principalement à afficher le formulaire et à transmettre les éventuelles informations d’erreur.

La gestion de l’authentification est effectuée par le composant Security de Symfony.



Firewall

[Configuration](#)[Parcours](#)

Configuration

Le firewall principal est configuré dans :

```
config/packages/security.yaml
```

La configuration est :

```
main:
  lazy: true
  pattern: ^/
```

Le firewall main couvre donc les routes principales de l'application.

Il utilise le système de formulaire Symfony :

```
form_login:
  login_path: login
  check_path: login
  enable_csrf: true
  csrf_token_id: authenticate
```

La route utilisée pour afficher le formulaire et traiter l'authentification est :

```
/login
```

 **ToDo & Co** LE PROCESSUS D’AUTHENTIFICATION

 Firewall

- Configuration
-  **Parcours**

 Parcours

Le fonctionnement peut être résumé ainsi :





ToDo & Co

LE PROCESSUS D’AUTHENTIFICATION

 Déconnexion

-  Déconnexion
-  Parcours

 Déconnexion

La déconnexion est également configurée dans le firewall :

```
logout:
  path: logout
  target: /
```

Lorsqu’un utilisateur demande :

```
/logout
```

Symfony gère automatiquement la déconnexion.



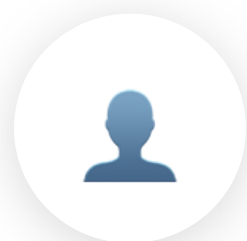
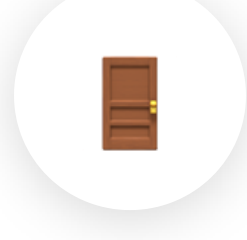
ToDo & Co

LE PROCESSUS D’AUTHENTIFICATION

Déconnexion

Parcours

Le parcours est :

Utilisateur connecté		
		
GET /logout		
		
Symfony Security		
		
Déconnexion		
		
Redirection vers /		

Après celle-ci, l'utilisateur est redirigé vers :

/



ToDo & Co

PROTECTION DES MOTS DE PASSE

Hashage des mots de passe

 Protection Utilisation Parcours

Protection

La configuration suivante définit le système utilisé pour protéger les mots de passe :

```
password_hashers:  
  App\Entity\User:  
    algorithm: auto
```

Symfony détermine automatiquement l'algorithme de hashage approprié.

Lorsqu'un utilisateur est créé ou que son mot de passe est modifié, le mot de passe ne doit donc pas être enregistré directement en clair dans la base de données.



ToDo & Co

PROTECTION DES MOTS DE PASSE

 Hashage des mots de passe

-  Protection

 Utilisation

 Parcours

 Utilisation

Dans UserController, le hashage est réalisé avec :

```
UserPasswordHasherInterface
```



ToDo & Co

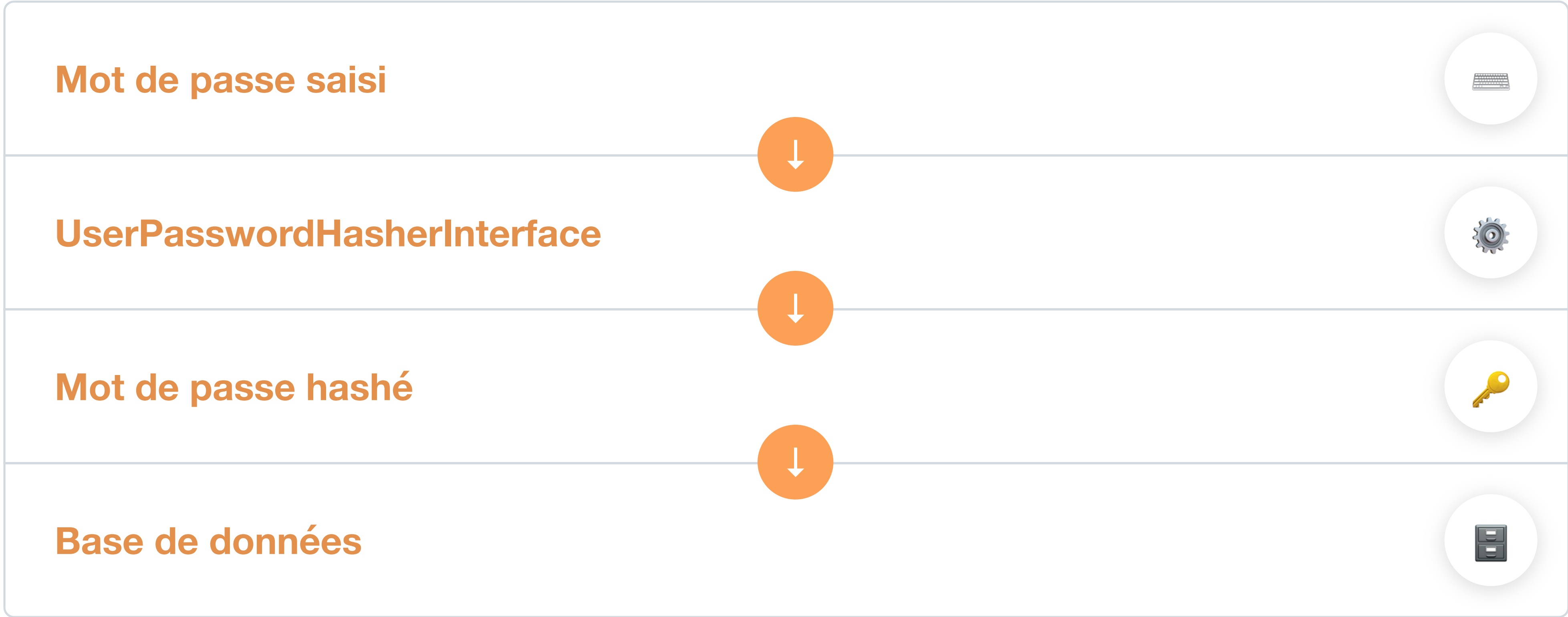
PROTECTION DES MOTS DE PASSE

🔑 Hashage des mots de passe

- 🔒 Protection
- 📖 Utilisation
- 🗺️ Parcours

🗺️ Parcours

Le principe est donc :



Lors de l'authentification, Symfony compare ensuite le mot de passe fourni avec le hash enregistré.

[← 🔑 Hashage des mots de passe 3](#)[🔑 Mot de passe dans l'entité User 2 →](#)

ToDo & Co

PROTECTION DES MOTS DE PASSE

🔑 Mot de passe dans l'entité User

[📖 Utilisation](#)[🛡️ Protection](#)

📖 Utilisation

L'entité User implémente :

```
PasswordAuthenticatedUserInterface
```

et fournit la méthode :

```
public function getPassword(): string
{
    return $this->password;
}
```

Symfony utilise cette méthode pour récupérer le hash du mot de passe enregistré.

Le mot de passe lui-même n'est pas stocké en clair.



ToDo & Co

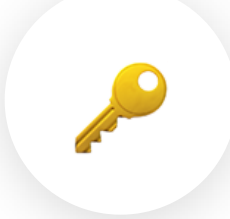
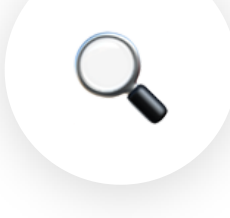
PROTECTION DES MOTS DE PASSE

 Mot de passe dans l'entité User

- Utilisation
-  Protection

 Protection

Le principe est donc :

Mot de passe		
	↓	
Hashage		
	↓	
Hash stocké		
	↓	
Vérification lors de la connexion		
	↓	
Authentification		

 **ToDo & Co** RÔLES ET CONTRÔLE DES ACCÈS

 Gestion des rôles

- Rôles
- Fonctionnement

 Rôles

L'application utilise deux rôles :

ROLE_USER
ROLE_ADMIN

Dans l'entité User, la méthode :

getRoles()

garantit qu'un utilisateur possède toujours au minimum :

ROLE_USER



ToDo & Co

RÔLES ET CONTRÔLE DES ACCÈS

Gestion des rôles

- 

Rôles
- 

Fonctionnement

Fonctionnement

Par exemple, si l'utilisateur possède :

```
[ 'ROLE_ADMIN' ]
```

la méthode retourne :

```
ROLE_ADMIN  
ROLE_USER
```

Le rôle administrateur bénéficie donc également des permissions accordées à ROLE_USER.

[← Gestion des rôles 2](#)[Modification du rôle 2 →](#)

ToDo & Co

RÔLES ET CONTRÔLE DES ACCÈS

Modification du rôle

[Rôles](#)[Validation](#)

Rôles

Le rôle peut être défini avec :

```
setRole()
```

La méthode accepte uniquement :

```
ROLE_USER  
ROLE_ADMIN
```

Toute autre valeur provoque une :

```
InvalidArgumentException
```

Cela permet de limiter les rôles disponibles aux deux rôles prévus par l'application.

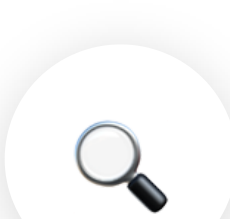
 **ToDo & Co** RÔLES ET CONTRÔLE DES ACCÈS

 Modification du rôle

-  Rôles
-  Validation

 Validation

Le principe est donc :

Rôle sélectionné		
	↓	
setRole()		
	↓	
Vérification de la valeur		
	↓	
ROLE_USER / ROLE_ADMIN		
	↓	
Rôle enregistré		

🚦

Contrôle des accès

⚙️

Configuration

Les règles d'autorisation sont définies avec :

```
access_control:
```

La configuration actuelle est :

```
access_control:
- { path: ^/login, roles: PUBLIC_ACCESS }
- { path: ^/users, roles: ROLE_ADMIN }
- { path: ^/, roles: ROLE_USER }
```

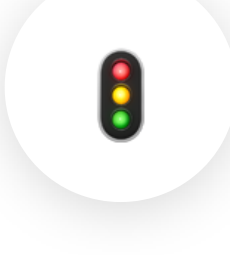
Ces règles sont évaluées pour déterminer si l'utilisateur peut accéder à une route.

 **ToDo & Co** RÔLES ET CONTRÔLE DES ACCÈS

 Contrôle des accès

- Configuration
-  **Parcours — Accès administrateur**
-  **Parcours — Accès utilisateur**

 Parcours — Accès administrateur

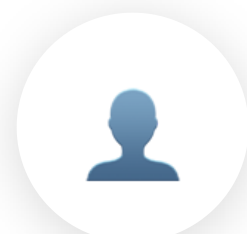
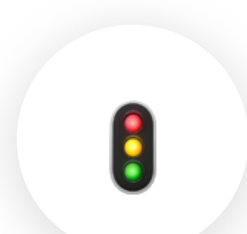
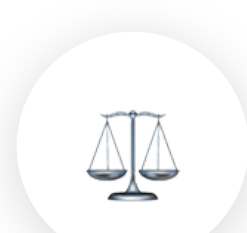
ROLE_ADMIN		
	↓	
/users		
	↓	
access_control		
	↓	
Vérification du rôle		
	↓	
Accès autorisé		

 **ToDo & Co** RÔLES ET CONTRÔLE DES ACCÈS

 Contrôle des accès

- Configuration
-  **Parcours — Accès administrateur**
-  **Parcours — Accès utilisateur**

 Parcours — Accès utilisateur

ROLE_USER		
	↓	
/users		
	↓	
access_control		
	↓	
Vérification du rôle		
	↓	
Access Denied		

 **ToDo & Co** RÔLES ET CONTRÔLE DES ACCÈS

 Accès à /login

 Route publique

 Parcours

 Route publique

La route :

/login

utilise :

PUBLIC_ACCESS

Elle peut donc être consultée sans être authentifié.



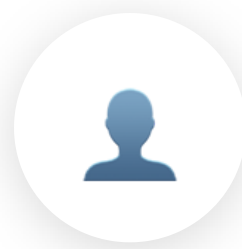
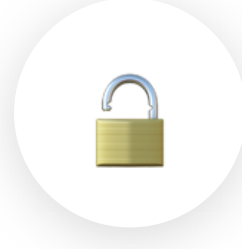
ToDo & Co

RÔLES ET CONTRÔLE DES ACCÈS

 Accès à /login

 Parcours

C'est nécessaire puisque l'utilisateur doit pouvoir accéder à la page de connexion avant d'être authentifié.

Utilisateur non authentifié		
		
GET /login		
		
PUBLIC_ACCESS		
		
Page accessible		



ToDo & Co

RÔLES ET CONTRÔLE DES ACCÈS



Gestion des utilisateurs

-  Administrateur
-  Autorisation



Administrateur

Les routes commençant par :

/users

nécessitent :

ROLE_ADMIN

 **ToDo & Co** RÔLES ET CONTRÔLE DES ACCÈS

 Gestion des utilisateurs

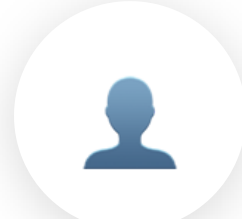
-  Administrateur
-  **Autorisation**

 Autorisation

Ainsi :

ROLE_ADMIN		
	↓	
/users		
	↓	
Accès autorisé		

alors que :

ROLE_USER		
	↓	
/users		
	↓	
Accès autorisé		

Cette règle protège notamment les fonctionnalités de gestion des utilisateurs.



ToDo & Co

RÔLES ET CONTRÔLE DES ACCÈS

 Accès aux autres pages

 Utilisateur

 Accès

 Utilisateur

La règle :

```
- { path: ^/, roles: ROLE_USER }
```

protège toutes les autres routes correspondant au chemin /.



ToDo & Co

RÔLES ET CONTRÔLE DES ACCÈS

 Accès aux autres pages

-  Utilisateur
-  Accès

 Accès

Un utilisateur doit donc être authentifié et disposer du rôle :

ROLE_USER

pour accéder aux fonctionnalités normales de l'application.

 **ToDo & Co** AUTORISATION DES ACTIONS SUR LES TÂCHES

Authentification vs autorisation

Il est important de distinguer les deux notions.

 **Authentification**

 **Autorisation**

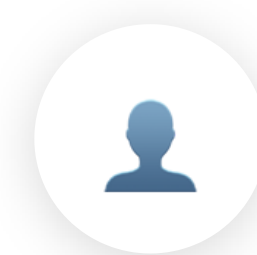
 **Parcours**

Authentification

Elle répond à la question :

« Qui est l'utilisateur ? »

Elle est principalement gérée par :

- Symfony Security** 
- Provider Doctrine** 
- User** 
- Firewall** 

 **ToDo & Co** AUTORISATION DES ACTIONS SUR LES TÂCHES

 Authentification vs autorisation

Il est important de distinguer les deux notions.

 Authentification

 **Autorisation**

 Parcours

 Autorisation

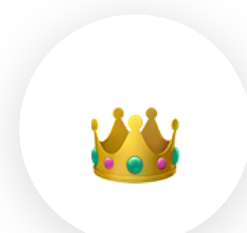
Elle répond à la question :

« Cet utilisateur a-t-il le droit d’effectuer cette action ? »

Elle repose notamment sur :

- access_control


- rôles


- TaskVoter





ToDo & Co

AUTORISATION DES ACTIONS SUR LES TÂCHES



Authentification vs autorisation

Il est important de distinguer les deux notions.

 Authentification

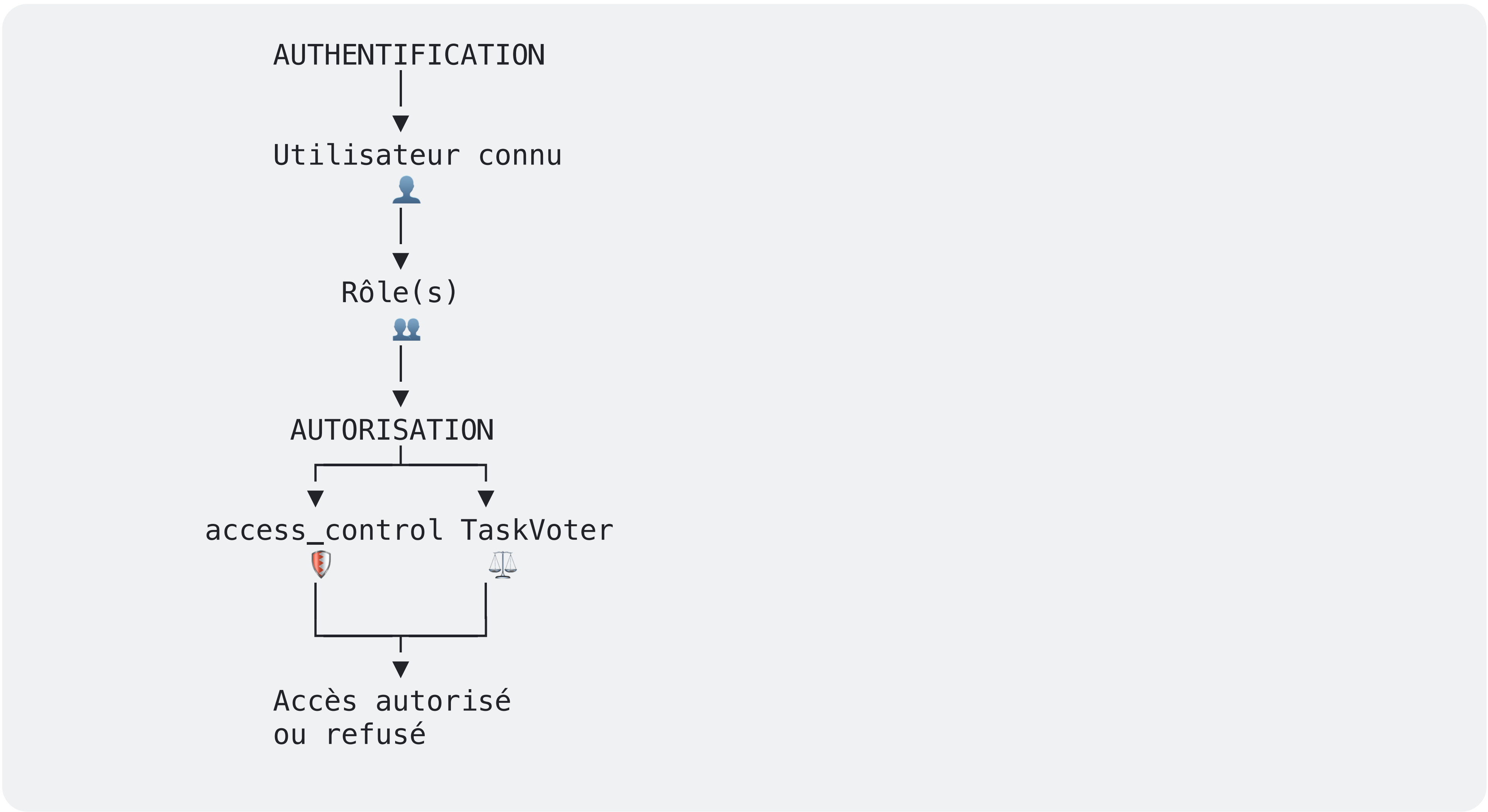
 Autorisation

 Parcours



Parcours

Le fonctionnement global peut être résumé ainsi :



 **ToDo & Co** AUTORISATION DES ACTIONS SUR LES TÂCHES

 TaskVoter

-  **Fonctionnement**
-  **Décision**

 Fonctionnement

Pour les tâches, les décisions sont prises notamment par :

```
src/Security/Voter/TaskVoter.php
```

Le Voter prend en charge :

```
EDIT  
TOGGLE  
DELETE
```

 **ToDo & Co**

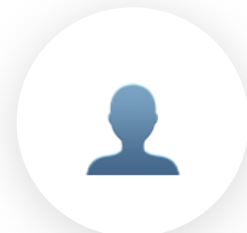
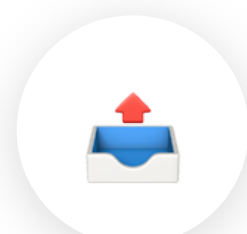
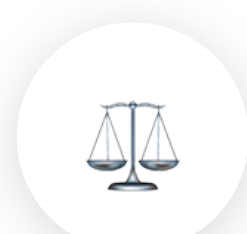
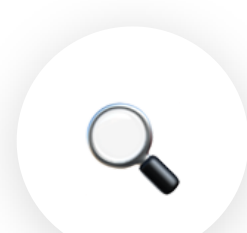
AUTORISATION DES ACTIONS SUR LES TÂCHES

 TaskVoter

- Fonctionnement
-  **Décision**

 Décision

Le principe est donc :

Utilisateur authentifié	↓	
Action sur une Task	↓	
TaskVoter	↓	
Vérification des droits	↓	
GRANTED / DENIED		

Le propriétaire peut notamment agir sur sa propre tâche.

Un autre utilisateur ne peut pas modifier ou supprimer une tâche qui ne lui appartient pas.



ToDo & Co

AUTORISATION DES ACTIONS SUR LES TÂCHES

 Règle pour anonymous

-  Utilisateur anonyme
-  Règle spéciale

 Utilisateur anonyme

Les tâches associées à :

anonymous

possèdent une règle particulière.

Leur suppression est réservée aux administrateurs.

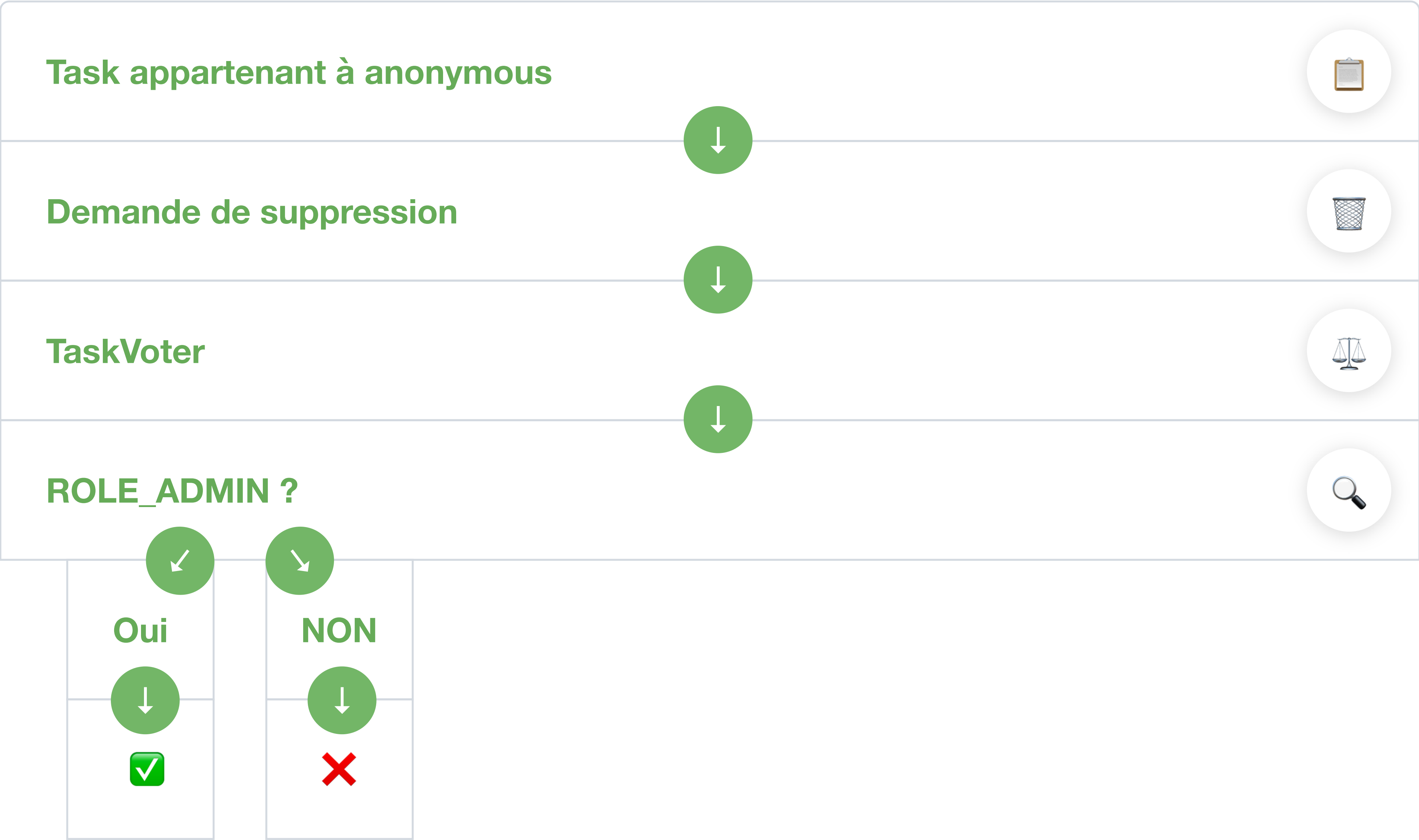
 **ToDo & Co** AUTORISATION DES ACTIONS SUR LES TÂCHES

 Règle pour anonymous

-  Utilisateur anonyme
-  **Règle spéciale**

 Règle spéciale

Le principe est donc :



Cette règle permet de conserver les anciennes tâches tout en protégeant leur suppression.



ToDo & Co

AUTORISATION DES ACTIONS SUR LES TÂCHES

Règles de suppression

Autorisation

Autorisation

Les règles de suppression peuvent être résumées ainsi :

Situation	Résultat
Utilisateur propriétaire de la tâche	 Suppression autorisée
Utilisateur non propriétaire	 Suppression refusée
Tâche appartenant à anonymous + ROLE_USER	 Suppression refusée
Tâche appartenant à anonymous + ROLE_ADMIN	 Suppression autorisée



ToDo & Co

FAIRE ÉVOLUER LA SÉCURITÉ DE L'APPLICATION

 Modifier le comportement de connexion

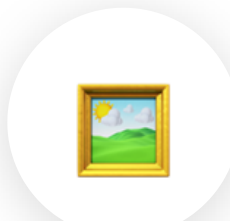
Fichiers |  Évolution

Fichiers |  Évolution

Consulter principalement :

config/packages/security.yaml

src/Controller/SecurityController.php

templates/security/login.html.twig

  Modifier le comportement de connexion

 Modifier les rôles disponibles 

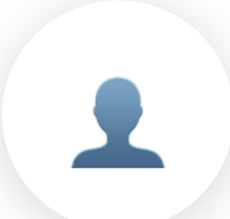
 **ToDo & Co** FAIRE ÉVOLUER LA SÉCURITÉ DE L'APPLICATION

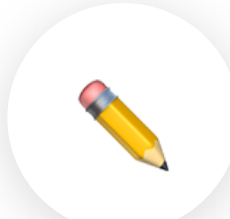
 Modifier les informations d'un utilisateur

 Fichiers |  Évolution

 **Fichiers |  Évolution**

Consulter :

src/Entity/User.php

src/Form/UserType.php

src/Controller/UserController.php

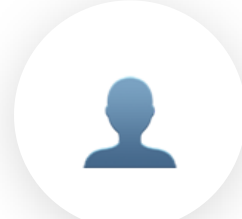
 **ToDo & Co** FAIRE ÉVOLUER LA SÉCURITÉ DE L'APPLICATION

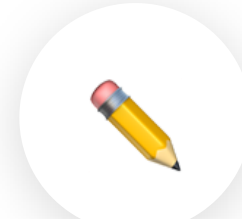
 Modifier les rôles disponibles

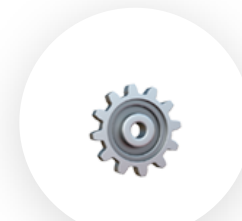
 Fichiers |  Évolution

 **Fichiers |  Évolution**

Consulter :

src/Entity/User.php

src/Form/UserType.php

config/packages/security.yaml

et vérifier également les tests associés.

←  Modifier les rôles disponibles

 Modifier les accès aux routes →



ToDo & Co

FAIRE ÉVOLUER LA SÉCURITÉ DE L'APPLICATION

 Modifier les autorisations sur les tâches

 Fichiers |  Évolution

 **Fichiers |  Évolution**

Consulter :

src/Security/Voter/TaskVoter.php

et les tests :

tests/Security/Voter/TaskVoterTest.php

77



ToDo & Co

FAIRE ÉVOLUER LA SÉCURITÉ DE L'APPLICATION

 Modifier les accès aux routes

 Fichiers |  Évolution

 **Fichiers |  Évolution**

Consulter :

`config/packages/security.yaml`



Les modifications de sécurité doivent toujours être accompagnées de tests permettant de vérifier les accès autorisés et refusés.



ToDo & Co

TESTS ET VÉRIFICATION DE LA SÉCURITÉ

 Tests fonctionnels

 Cas testés

 Cas testés

L'authentification et les autorisations sont couvertes par les tests fonctionnels.

Les tests permettent notamment de vérifier :

- l'accès à /login


- la connexion avec un utilisateur valide


- le refus d'identifiants invalides





ToDo & Co

TESTS ET VÉRIFICATION DE LA SÉCURITÉ

 Tests fonctionnels

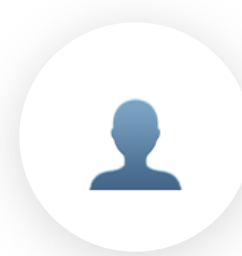
 Cas testés

 Cas testés

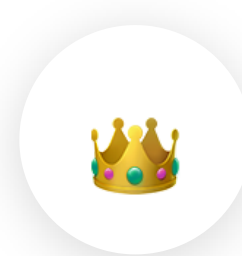
la redirection d'un utilisateur non authentifié



l'accès d'un ROLE_USER aux fonctionnalités normales



l'accès d'un ROLE_ADMIN à la gestion des utilisateurs



le refus d'accès d'un ROLE_USER à /users





ToDo & Co

TESTS ET VÉRIFICATION DE LA SÉCURITÉ

 Tests du TaskVoter

-  Cas testés
-  Autorisations

 Cas testés

Les tests du TaskVoter vérifient les autorisations spécifiques aux tâches.

Les principales actions testées sont :

EDIT
TOGGLE
DELETE



ToDo & Co

TESTS ET VÉRIFICATION DE LA SÉCURITÉ

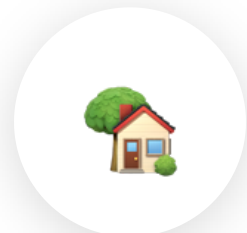
 Tests du TaskVoter

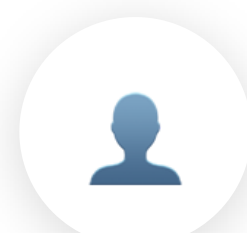
- Cas testés
-  Autorisations

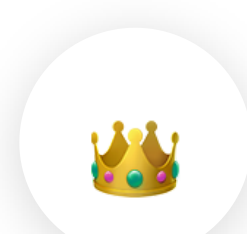
 Autorisations

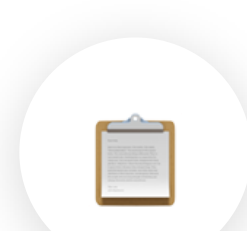
Les tests couvrent notamment :

- le propriétaire de la tâche


- un autre utilisateur


- un administrateur


- une tâche appartenant à anonymous





ToDo & Co

TESTS ET VÉRIFICATION DE LA SÉCURITÉ

 Exécution des tests

 Commande

Les tests sont exécutés avec :

```
vendor/bin/phpunit
```

Les tests permettent de vérifier que les règles de sécurité restent conformes aux besoins fonctionnels après une modification du code.



ToDo & Co

TESTS ET VÉRIFICATION DE LA SÉCURITÉ

▶

Exécution des tests

Résultat

```
• justineleleu@MacBook-Pro-de-factoria ToDo-and-Co % docker compose exec php vendor/bin/phpunit
PHPUnit 11.5.56 by Sebastian Bergmann and contributors.

Runtime:      PHP 8.2.33
Configuration: /var/www/html/phpunit.xml.dist

.....                                     56 / 56 (100%)

Time: 00:04.465, Memory: 46.50 MB

OK (56 tests, 123 assertions)
❖ justineleleu@MacBook-Pro-de-factoria ToDo-and-Co %
```



ToDo & Co

RÉSUMÉ POUR UN NOUVEAU DÉVELOPPEUR

 À retenir

Pour comprendre rapidement l’authentification du projet, retenir les éléments suivants :

- 1
- 2
- 3
- 4
- 5
- 6
- 7

 User.php

User.php

↓

Représente l'utilisateur Symfony



ToDo & Co

RÉSUMÉ POUR UN NOUVEAU DÉVELOPPEUR

📌 À retenir

Pour comprendre rapidement l’authentification du projet, retenir les éléments suivants :

- 1
- 2
- 3
- 4
- 5
- 6
- 7

 **security.yaml**

security.yaml

↓

Configure Security, le provider, le firewall et les accès






ToDo & Co

RÉSUMÉ POUR UN NOUVEAU DÉVELOPPEUR

📌 À retenir

■ Pour comprendre rapidement l’authentification du projet, retenir les éléments suivants :

- 1
- 2
- 3
- 4
- 5
- 6
- 7

🎮 SecurityController.php

SecurityController.php



Affiche la page de connexion





ToDo & Co

RÉSUMÉ POUR UN NOUVEAU DÉVELOPPEUR

📌

À retenir

■ Pour comprendre rapidement l’authentification du projet, retenir les éléments suivants :

- 1
- 2
- 3
- 4
- 5
- 6
- 7



login.html.twig

login.html.twig



Contient le formulaire de connexion





ToDo & Co

RÉSUMÉ POUR UN NOUVEAU DÉVELOPPEUR

À retenir

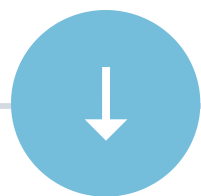
Pour comprendre rapidement l’authentification du projet, retenir les éléments suivants :

- 1
- 2
- 3
- 4
- 5
- 6
- 7



UserPasswordHasherInterface

UserPasswordHasherInterface



Protège les mots de passe





ToDo & Co

RÉSUMÉ POUR UN NOUVEAU DÉVELOPPEUR

📌

À retenir

■ Pour comprendre rapidement l’authentification du projet, retenir les éléments suivants :

- 1
- 2
- 3
- 4
- 5
- 6
- 7



access_control

UserPasswordHasherInterface



Protège les routes selon les rôles





ToDo & Co

RÉSUMÉ POUR UN NOUVEAU DÉVELOPPEUR

 À retenir

■ Pour comprendre rapidement l’authentification du projet, retenir les éléments suivants :

- 1
- 2
- 3
- 4
- 5
- 6
- 7

 TaskVoter

TaskVoter

↓

Protège les actions sur les tâches





ToDo & Co

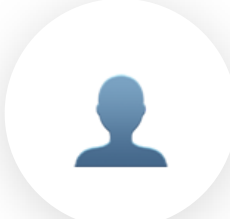
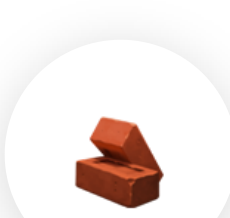
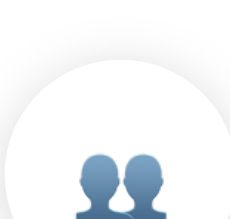
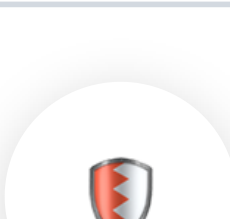
RÉSUMÉ POUR UN NOUVEAU DÉVELOPPEUR

 Parcours global

-  Général
-  Règles de sécurité
-  Évolution de la sécurité

 Général

Le principe général est donc :

Utilisateur		
	↓	
Authentification Symfony		
	↓	
User chargé depuis Doctrine		
	↓	
Rôles récupérés		
	↓	
Contrôle des accès		
	↓	
Action autorisée ou refusée		 / 



ToDo & Co

RÉSUMÉ POUR UN NOUVEAU DÉVELOPPEUR

 Parcours global

-  Général
-  Règles de sécurité
-  Évolution de la sécurité

 Règles de sécurité

Cette architecture permet de centraliser les règles de sécurité tout en séparant clairement :

Authentification	↓	
Gestion des utilisateurs	↓	
Gestion des rôles	↓	
Contrôle des routes	↓	
Autorisations métier	↓	
Tests	↓	



ToDo & Co

RÉSUMÉ POUR UN NOUVEAU DÉVELOPPEUR



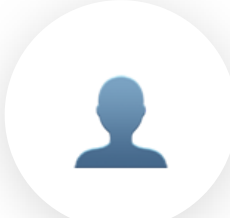
Parcours global

-  Général
-  Règles de sécurité
-  **Évolution de la sécurité**



Évolution de la sécurité

Pour une évolution de la sécurité, il est donc essentiel d’identifier le niveau concerné avant de modifier le code.

Connexion ?	
↓	
 security.yaml /  SecurityController /  login.html.twig	
Utilisateur ?	
↓	
 User.php /  UserType.php /  UserController.php	



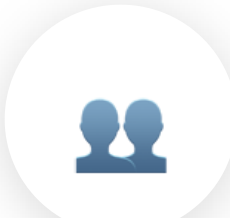
ToDo & Co

RÉSUMÉ POUR UN NOUVEAU DÉVELOPPEUR

 Parcours global

-  Général
-  Règles de sécurité
-  Évolution de la sécurité

 Évolution de la sécurité

Rôle ?	
↓	
 User.php /  UserType.php /  security.yaml	
Route ?	
↓	
security.yaml	



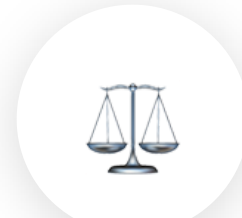
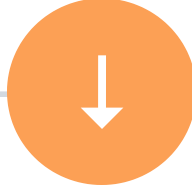
ToDo & Co

RÉSUMÉ POUR UN NOUVEAU DÉVELOPPEUR

 Parcours global

-  Général
-  Règles de sécurité
-  **Évolution de la sécurité**

 Évolution de la sécurité

Action sur une tâche ?		
		
TaskVoter.php		
Vérification ?		
		
Tests		

La séparation de ces responsabilités facilite la compréhension, la maintenance et l'évolution de l'application.